ジオメトリック・インテリジェンス

ケーススタディ補助文書 03

実装スクリプト集

― 白夜の臨界：ジオメトリック・インテリジェンス理論の北極海LNG保険リスクへの適用 ―

本書は、ケーススタディ・ノベル「白夜の臨界」の数学について、
実行して出力を確かめた最小限の参照実装と、実装上の注意を示す。

Étale Cohomology

2026 年 11 月

License: CC BY 4.0

目次

1　はじめに　2

2　臨界確率 κ_c（まえがき、第９章）　2

2.1　ブートストラップによる区間　2

3　正則化（まえがき、第３章）　3

4　シンプソンのパラドックスとスキャン検定（第１５章）　3

5　並べ替え検定（第１４章・第１６章）　4

6　レイリー検定（第１６章）　5

7　季節のデータの操作の検出（付属資料のコード）　5

8　整合性の半径（第６章）　5

9　付属資料のコード　6

第1章　はじめに

本書のコードは、本作で登場する考え方を、手元で確かめられる最小限の例である。すべてPython（NumPy、SciPy）で実行し、出力を確認した。第 2 章の κ_c の計算は、本編のパラメータで本編の数値を再現する。それ以外の章のデータは、手順を示すための説明用の数値であり、本編の数値そのものを再現するものではない。

本編のまえがきに掲載したコード（核心部分）の全文は、GitHubに掲載する付属資料のPythonファイル「白夜の臨界_付属資料_保険リスク多様体の構築.py」にある（第９章）。

第2章　臨界確率 κ_c（まえがき、第９章）

一般化パレート分布で1回の航海あたりの確率 p を求め、1シーズン（88回）に積み上げる。形状パラメータ ξ を変えたときの感度も示す。

```
from scipy.stats import genpareto
# 臨界確率 κ_c：1シーズンの航海の中で、少なくとも1回、損害額が保険プールの引受キャパシティ C を超える確率
u, xi, sigma = 50e6, 0.31, 180e6   # 閾値5000万ドル、一般化パレート分布の形状・尺度
zeta = 0.043                       # 損害額が閾値 u を超える割合
C, N = 620e6, 88                   # 引受キャパシティ6.2億ドル、1シーズンの航海の回数
p = zeta * genpareto.sf(C - u, xi, scale=sigma)   # 1回の航海あたり
kappa = 1 - (1 - p) ** N                           # 1シーズン全体
print(f"p = {p:.4f}   kappa_c = {kappa:.3f}")
for xi_ in (0.0, 0.2, 0.31, 0.5):                  # 裾の重さ ξ を変えると
    p_ = zeta * genpareto.sf(C - u, xi_, scale=sigma)
    print(f"xi = {xi_:.2f}: p = {p_:.5f}, kappa_c = {1-(1-p_)**N:.3f}")
```

出力：

```
p = 0.0047   kappa_c = 0.341
xi = 0.00: p = 0.00181, kappa_c = 0.148
xi = 0.20: p = 0.00370, kappa_c = 0.278
xi = 0.31: p = 0.00473, kappa_c = 0.341
xi = 0.50: p = 0.00644, kappa_c = 0.434
```

読み方：本編の $p \approx 0.0047$、$\kappa_c \approx 0.34$ を再現する。裾が軽い（$\xi = 0$）と0.15、重い（$\xi = 0.5$）と0.43になり、裾の重さの推定が結論を大きく左右する。

2.1　ブートストラップによる区間

```
import numpy as np
from scipy.stats import genpareto
# ブートストラップ：少ない超過データから推定した ξ, σ と κ_c の幅（説明用の合成データ）
rng = np.random.default_rng(1)
u, zeta, C, N = 50e6, 0.043, 620e6, 88
true_xi, true_sigma = 0.31, 180e6
excess = genpareto.rvs(true_xi, scale=true_sigma, size=40, random_state=rng)  # 閾値超過の損害（合成）
def kappa_from(sample):
```

```
    xi, loc, sigma = genpareto.fit(sample, floc=0)
    p = zeta * genpareto.sf(C - u, xi, scale=sigma)
    return 1 - (1 - p) ** N
k_hat = kappa_from(excess)
boots = [kappa_from(rng.choice(excess, size=len(excess), replace=True)) for _ in range(500)]
lo, hi = np.percentile(boots, [2.5, 97.5])
print(f"点推定 kappa_c = {k_hat:.3f}　ブートストラップ95%区間 = [{lo:.3f}, {hi:.3f}]
↪  （超過データ{len(excess)}件）")
```

出力：

```
点推定 kappa_c = 0.360　ブートストラップ95%区間 = [0.175, 0.539]　（超過データ40件）
```

実装上の注意：説明用の合成データ（超過データ40件）で手順を示した。超過データが少ないほど区間は広くなる。本編の95%区間 0.22〜0.47 は、本編の設定（実データとシミュレーションの組み合わせ）による値である。

第3章　正則化（まえがき、第３章）

```
import numpy as np
# 正則化：退化しかけた行列に λI を加えると、固有値がすべて λ 以上になる
rng = np.random.default_rng(0)
g = rng.normal(size=(7, 8))            # 7件のデータから作る8次元の経験フィッシャー行列（階数は最大7）
F = g.T @ g / 7
lam = 1e-3
print("正則化前の最小固有値:", f"{np.linalg.eigvalsh(F).min():.2e}")
print("正則化後の最小固有値:", f"{np.linalg.eigvalsh(F + lam*np.eye(8)).min():.2e}")
```

出力：

```
正則化前の最小固有値: 4.14e-17
正則化後の最小固有値: 1.00e-03
```

読み方：7件のデータから作る8次元の行列は階数が足りず、最小固有値はほぼゼロ（退化）。λI を加えると、最小固有値はちょうど λ になり、計量の正定値性が回復する。λ が大きすぎると、データの情報を均してしまう。

第4章　シンプソンのパラドックスとスキャン検定（第１５章）

```
import numpy as np
# シンプソンのパラドックス：船の区分ごとには事故率が上がっているのに、全体では下がって見える
#  （記録の期間に、新しい砕氷船の航海の割合が増えたため）
#  （区分：新しい砕氷船 / 古い船。数字は説明用）
before = {"新しい砕氷船": (2, 400), "古い船": (6, 100)}   # (事故件数, 航海数)
after  = {"新しい砕氷船": (3, 450), "古い船": (4, 50)}
for name in before:
    (a, n), (b, m) = before[name], after[name]
```

```
    print(f"{name}: 前 {a/n:.3f} → 後 {b/m:.3f}")
tb = sum(a for a, _ in before.values()) / sum(n for _, n in before.values())
ta = sum(a for a, _ in after.values()) / sum(n for _, n in after.values())
print(f"全体: 前 {tb:.3f} → 後 {ta:.3f}")
```

出力：

```
新しい砕氷船: 前 0.005 → 後 0.007
古い船: 前 0.060 → 後 0.080
全体: 前 0.016 → 後 0.014
```

読み方：どちらの区分でも事故率は上がっているのに、新しい砕氷船の航海の割合が増えたため、全体では下がって見える。

```
import numpy as np
# スキャン統計：ある長さの記録の中に「事故のない2年間」が偶然見つかる確率を、選んだことを考慮して評価する
rng = np.random.default_rng(0)
years, rate = 20, 0.6            # 20年分の記録、各年に事故が起きる確率（説明用）
def has_clean_window(x, w=2):
    return any(x[i:i+w].sum() == 0 for i in range(len(x) - w + 1))
sims = [has_clean_window(rng.random(years) < rate) for _ in range(20000)]
print(f"事故のない2年間が、どこかに偶然見つかる確率 = {np.mean(sims):.3f}")
print(f"（特定の2年を事前に決めた場合の確率 = {(1-rate)**2:.3f}）")
```

出力：

```
事故のない2年間が、どこかに偶然見つかる確率 = 0.925
（特定の2年を事前に決めた場合の確率 = 0.160）
```

読み方：特定の2年を事前に決めれば、事故のない確率は16%にすぎない。しかし、20年の記録から都合のよい2年を後から探せば、93%の確率で見つかる。「事故のない2年間」は、選んだこと自体を考慮しないと、リスクの低下の証拠にならない。

第5章　並べ替え検定（第１４章・第１６章）

```
import numpy as np
# 並べ替え検定：4回のリセットのすべてで、直前3日以内にレポートが届く確率（偶然なら）
rng = np.random.default_rng(0)
days = 120                                  # 7月〜10月（約120日）
resets = np.array([20, 50, 81, 111])        # リセットの日（約30日周期）
reports = resets - np.array([2, 1, 3, 2])   # 実際のレポートの到着日（すべて直前3日以内）
def all_within(rep):
    return all(((r - rep >= 0) & (r - rep <= 3)).any() for r in resets)
obs = all_within(reports)
hits = sum(all_within(rng.choice(days, size=4, replace=False)) for _ in range(100000))
print(f"実際のデータで成り立つか: {obs}   偶然に成り立つ確率 ≈ {hits/100000:.5f}")
```

出力：

```
実際のデータで成り立つか: True　偶然に成り立つ確率 ≈ 0.00002
```

実装上の注意：時間とともに増える2つの日付の並びの相関係数は、関係がなくても1に近くなるので、連動の証拠には使わない。ここでは「4回すべてで、直前3日以内に届く」という出来事が、到着日をランダムに並べ替えたときに偶然どのくらい起きるかを調べている。

第6章　レイリー検定（第１６章）

```
import numpy as np
# レイリー検定：出来事の時刻が、ある周期（約30日）の同じ位相に集中しているか
from math import exp
times = np.array([18, 49, 78, 109])          # レポートの到着日
T = 30.5
theta = 2 * np.pi * (times % T) / T
n = len(theta); R = abs(np.exp(1j * theta).sum()) / n
Z = n * R**2
p = exp(-Z) * (1 + (2*Z - Z**2) / (4*n) - (24*Z - 132*Z**2 + 76*Z**3 - 9*Z**4) / (288*n**2))
print(f"平均合成長 R = {R:.3f}　Z = {Z:.2f}　p ≈ {p:.4f}")
```

出力：

```
平均合成長 R = 0.993　Z = 3.95　p ≈ 0.0078
```

読み方：到着日が、約30日の周期の同じ位相に集中しているほど、平均合成長 R は1に近づく。標本が4件と少ないので、p値の近似には補正項を含めている。

第7章　季節のデータの操作の検出（付属資料のコード）

付属資料のコードでは、夏の事故の記録だけを外した「選ばれたデータ」を、並べ替え検定で検出する。単純に平均の比で判定すると、事故がまれなため、操作のないデータでも偶然の偏りで誤って検出してしまう。並べ替え検定は、その偶然の偏りの大きさをデータそのものから見積もる。付属資料のコードの実行結果は次のとおりである。

```
(a) 操作のないデータ（5,000件）　　p値 = 0.1634　判定: 検出なし
(b) 操作のないデータ（50,000件）　　p値 = 0.5522　判定: 検出なし
(c) 夏の事故だけを外したデータ（50,000件）　p値 = 0.0005　判定: 操作の疑い検出
```

実装上の注意：事故の件数が少ないと、操作があっても検出できない（検出力が低い）。まれな事故のデータは少なく、だからこそ「選ばれた数字」がまかり通る。

第8章　整合性の半径（第６章）

```
import numpy as np
# 整合性の半径（最も簡単な層：制限写像は恒等写像）。内側だけで測る場合と、外の記録とつないで測る場合
def radius(x, edges):
    return max(abs(x[a] - x[b]) for a, b in edges)
```

```
# ノード0〜3：架空の著者の論文が示す「事故率」、ノード4〜5：外の記録（実在のデータベース、航行記録）
x = np.array([0.010, 0.011, 0.010, 0.011, 0.045, 0.048])
inner = [(0,1),(1,2),(2,3),(0,3)]
outer = inner + [(0,4),(1,4),(2,5),(3,5)]
r_in, r_out = radius(x, inner), radius(x, outer)
print(f"内側だけで測った半径: {r_in:.3f}　外の記録とつないで測った半径: {r_out:.3f}（約{r_out/r_in:.0f}倍）")
```

出力：

```
内側だけで測った半径: 0.001　外の記録とつないで測った半径: 0.038（約38倍）
```

読み方：架空の著者が互いに引用し合う閉じたネットワークは、内側だけで測ると辻褄が合いすぎて半径はほぼゼロになる。外の記録とつなぐと、継ぎ目で食い違いが現れる。実際には、制限写像を行列とし、層ラプラシアンや整合性のフィルトレーションで計算する。Neural Sheaf Diffusionでは制限写像を学習する。

第9章　付属資料のコード

GitHubには、本編の保険リスク多様体の構築の全体（シミュレーションによるデータの拡張、主成分分析による直交化、フィッシャー情報行列と正則化、κ_c の計算、季節の分析と並べ替え検定による操作の検出）を、1つのPythonファイルとして掲載する。冒頭に、深層学習と極値理論を組み合わせる研究の参考文献を記している。教育用のコードであり、実際の保険の判断に用いるものではない。